

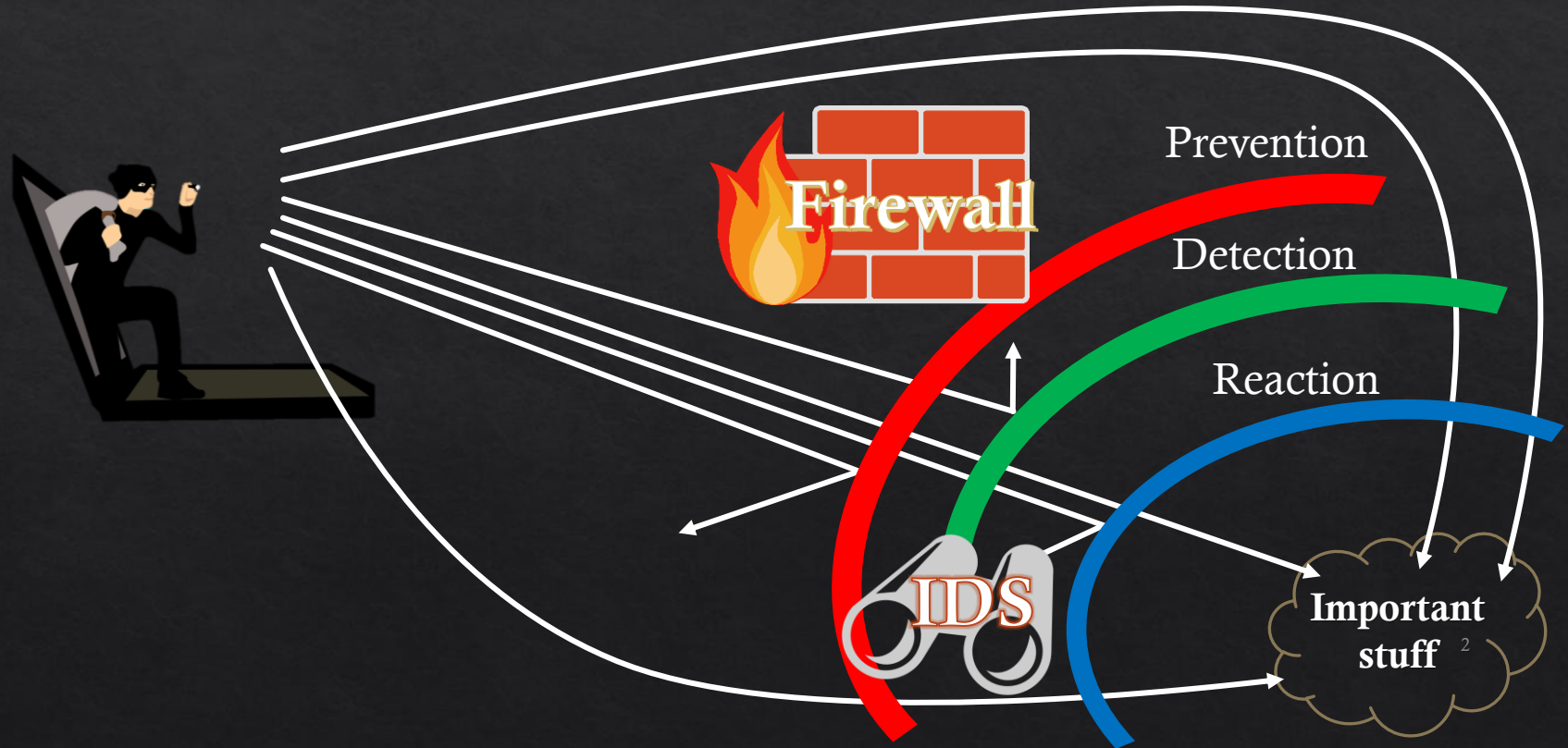


6. Security Tools

Garrison Gao

garrison.gao@uwa.edu.au

Tools



Tools and Methods

- ◇ Firewall
- ◇ Vulnerability (scanning)
- ◇ Sandbox

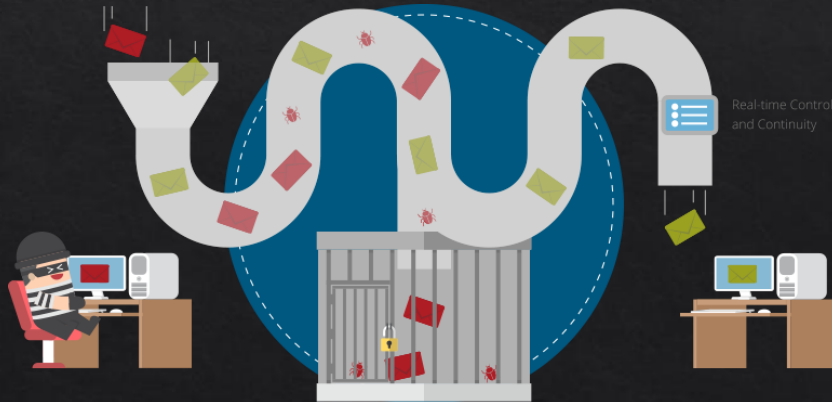
Firewall

- ◇ We create networks and subnets **everywhere**
 - ◇ Need to ensure that bad things don't come into our networks
- ◇ Firewalls are the foundations of a defence-in-depth network security strategy
- ◇ They are designed to protect organizations from network-based attacks
- ◇ The protection is achieved by filtering data packets that go through them (in and out)
 - ◇ Rules are set to check the traffic
- ◇ They can be a standalone network device (network-based hardware) or software (host-based software) on a computer system.



Firewall

- ◇ What are the things examined by a firewall?
 - ◇ IP address
 - ◇ Protocol headers, payload, or port number
 - ◇ Track of client-server sessions
 - ◇ Application level protocols



Types of Firewalls

- ◇ Packet filtering firewall
- ◇ Stateful firewall
- ◇ Application proxy firewall
- ◇ Stateful multilayer inspection firewall

Packet Filtering Firewalls

- ◆ First generation and most basic firewall
 - ◆ Also most inexpensive (hardware based) firewall, it is usually with high speed
- ◆ They inspect all data packets that attempt to traverse the firewall.
 - ◆ Based on pre-defined rules, packets are either allowed or denied
- ◆ These predefined rules are commonly called an Access Control List (ACL)
- ◆ They are considered as stateless firewalls
 - ◆ It analyzes each network packet in **isolation**, examining only header information (IPs, ports, protocols) against static rules, without tracking the **context**, **history**, or **session state** of connections. It does not know if a packet is part of an ongoing session

Packet Filtering Firewalls

- ◇ Packet filtering rules are common TCP/IP packet attributes
 - ◇ Source IP address
 - ◇ Destination IP address
 - ◇ IP protocol
 - ◇ Source TCP/UDP Port
 - ◇ Destination TCP/UDP Port
 - ◇ Inbound firewall network interface
 - ◇ Outbound firewall network interface

Packet Filtering Firewalls

- ◆ A list of rules are predefined
- ◆ If the incoming (or outgoing) packet matches to one of the rules, that rule is **invoked** (forward/discard)
- ◆ What happens when there is no matching rule?
 - ◆ A: invoke a default action
 - ◆ Default discard (everything is discarded)
 - ◆ Default forward (everything is allowed)

Packet Filtering Firewalls

Action	Ourhost	Port	Theirhost	Port	comment
Block	*	*	*	*	Default
Allow	*	25	*	*	Connect to SMTP
Block	*	*	badSMTP	*	Don't trust this host
allow	*	*	*	25	Connect to the mail server

We want to allow users to connect to our mailing server via SMTP at port 25.
We don't want users coming from *badSMTP*, what would be the rule to add?
We want to connect to their mail server via SMTP, what would be the rule to add?

Packet Filtering Firewalls

Action	Ourhost	Port	Theirhost	Port	comment
allow	*	*	*	25	Connect to the mail server

- ◇ Q: What could be a potential problem with this rule?
 - ◇ A: The restriction relies only on their port number, which cannot be controlled
 - ◇ An attacker can use the port to access internal machines

Packet Filtering Firewalls

Action	Ourhost	Port	Theirhost	Port	Flags	Comment
allow	*	*	*	25		Connect to the mail server

Allow	*	25	*	*	ACK	Check their reply (the mail server can only response to you after a connect is established, not initiate a new connection)
-------	---	----	---	---	-----	--

- ◆ Once a connection is set up, the ACK flag of a TCP segment is set to acknowledge segments sent from the other side
- ◆ The ACK signifies that the packet is part of an ongoing conversation
- ◆ It approximates a connection check, to be “stateful”

Packet Filtering Firewalls

- ◆ Advantages:

- ◆ Simple, low cost
- ◆ Typically transparent to users and are very fast

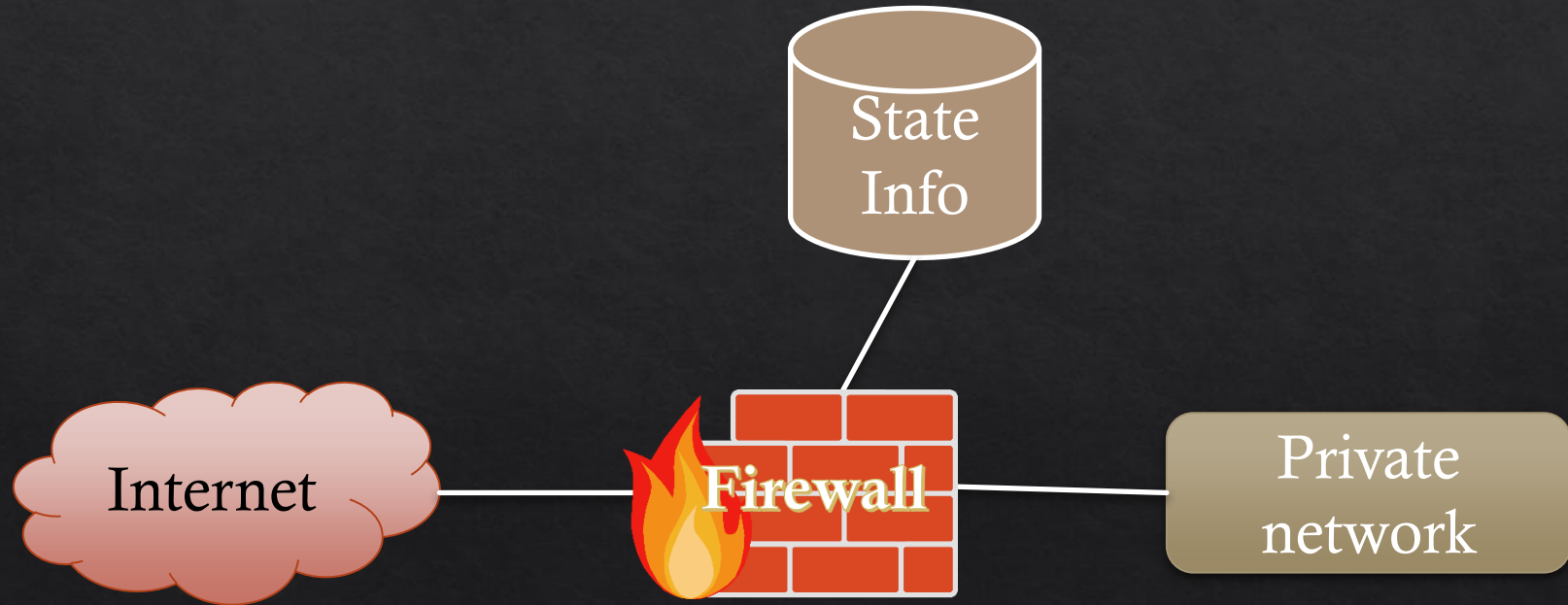
- ◆ Disadvantages:

- ◆ Cannot prevent attacks at the application layer
- ◆ Limited logging capabilities
- ◆ No advanced user authentication
- ◆ TCP/IP protocol bugs can be exploited – e.g., IP spoofing

Stateful Firewalls

- ◇ Generation 2 firewalls address this problem by examining each IP packet in context
 - ◇ Keeps the track of client-server sessions
 - ◇ Check each packet validity – e.g., does it belong to someone?
- ◇ Provides better capabilities to detect bogus packets

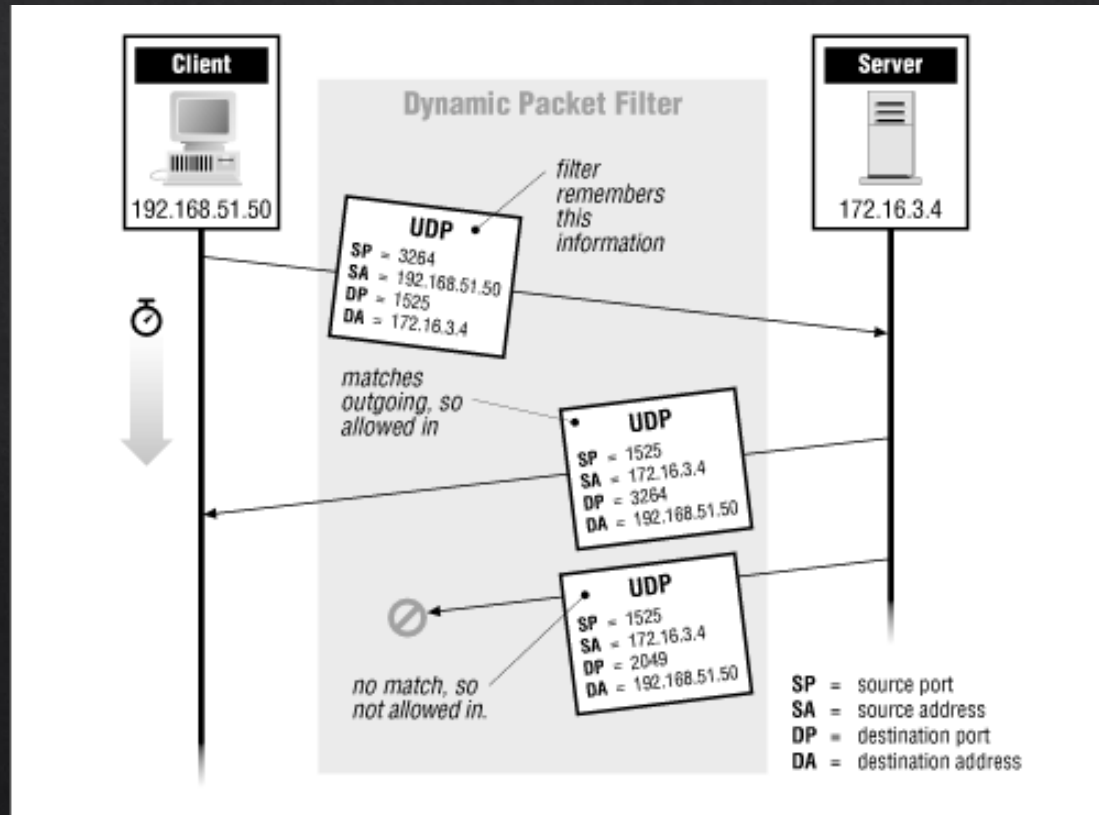
Stateful Firewalls



Stateful Firewalls

- ◇ TCP connections (typically)
 - ◇ Server port number < 1024
 - ◇ Client port number between 1024 and 16383
- ◇ Permanent assignments
 - ◇ (20, 21 -> FTP), (23 -> Telnet), (25 -> SMTP), (80 -> HTTP)
- ◇ If client wants to use port 2048, firewall must allow incoming traffic on this port
- ◇ Sol: To keep track of the outgoing requests

Stateful Firewalls



Stateful Firewalls

- ◇ The firewall keeps track of currently established connections

Source IP	Source Port	Dest. IP	Dest. Port	Con. State
192.168.1.100	1030	210.9.34.11	80	Established
192.168.1.103	1875	173.46.34.101	25	Established
192.43.1.101	2248	192.168.1.6	80	Established
210.168.1.113	1056	192.168.1.6	80	Established
222.99.1.73	1301	168.39.45.67	79	Established

the promie

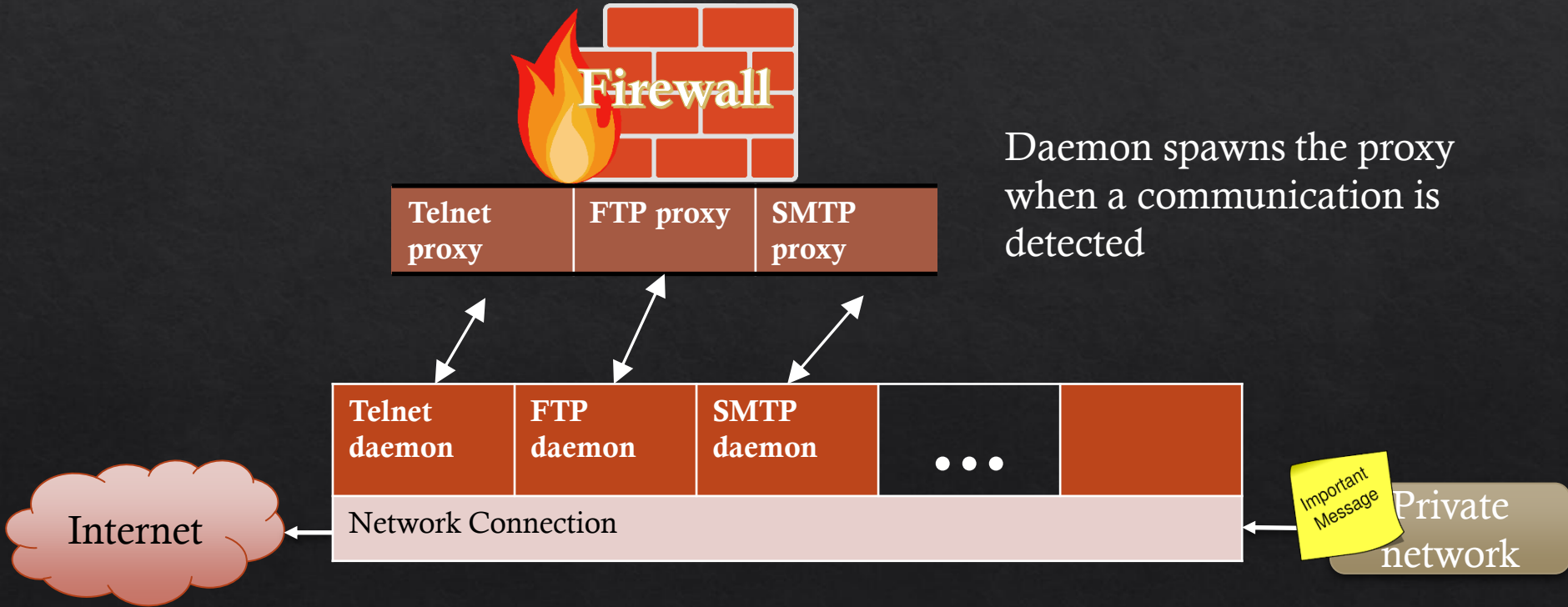
Stateful Firewalls

- ◇ Options to keep track of the TCP sequence numbers
 - ◇ To prevent attacks that depend on the sequence number
 - ◇ Q: What attack was it?
 - ◇ E.g., session hijacking
- ◇ Some may also inspect application data for well known protocols
 - ◇ e.g., FTP

Application-level Firewalls

- ◆ Also known as proxy servers, operate at the Application layer (layer 7) of the OSI model
- ◆ Specifically, proxy servers can provide the following services:
 - ◆ Filter: Filter packets based on an application or service (HTTPS, FTP, SMTP, etc)
 - ◆ For example, it prevents an employee from access social media websites within an organization
 - ◆ Caching
 - ◆ Logging

Application-level Firewalls

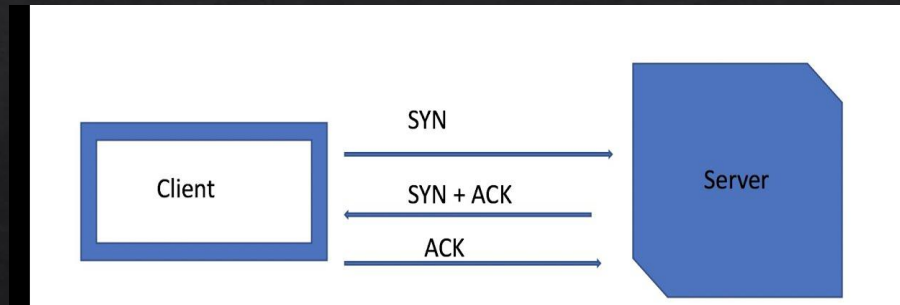


Application-level Firewalls

- ◆ Also known as proxy servers, operate at the Application layer (layer 7) of the OSI model
- ◆ Specifically, proxy servers can provide the following services:
 - ◆ Filter: Filter packets based on an application or service (HTTPS, FTP, SMTP, etc)
 - ◆ For example, it prevents an employee from access social media websites within an organization
 - ◆ Caching: Provides caching services
 - ◆ When you request a page from a website, the proxy server will retrieve it and then cache it in its memory
 - ◆ Th next time other user requests the same website, it will be retrieved from the cache
 - ◆ Benefit: saves bandwidth (e.g., video)
 - ◆ Logging: it can log user activity for auditing purpose

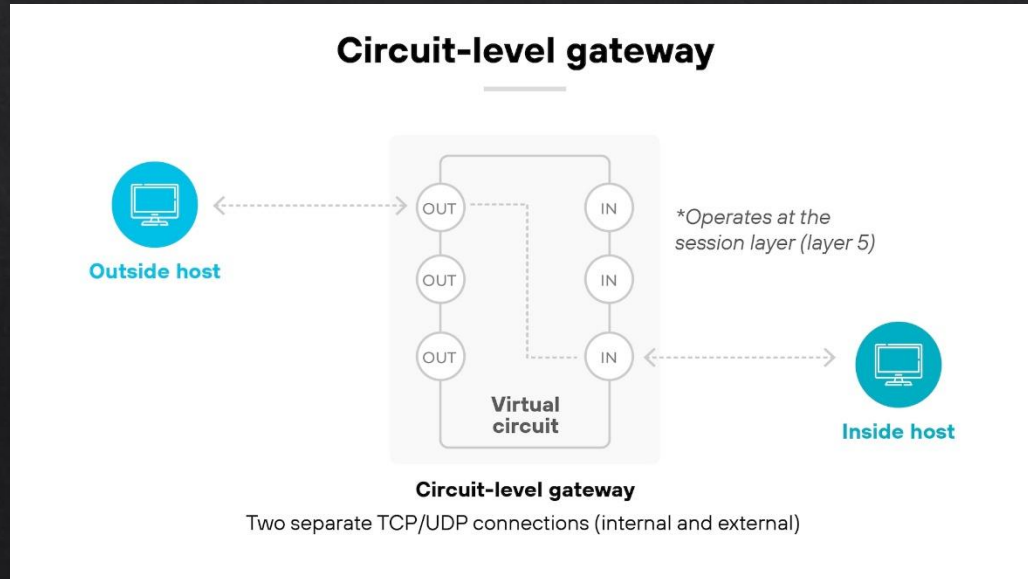
Circuit-level Firewalls

- ❖ Considered as **second generation** firewalls
- ❖ A circuit level gateway is a firewall that monitors and validates network sessions at the **session layer** (layer 5) of the OSI model.
- ❖ It can manage both TCP and UDP sessions, verifying that each connection follows the correct handshake or session initiation process.



Circuit-level Firewalls

- It establishes two separate TCP or UDP connections—one with the internal host and one with the external host—and relays packets only for approved sessions. It does not inspect packet contents but ensures that each connection follows the required handshake and security rules before data transfer occurs.



Circuit-level Firewalls

- ◇ Proxy at the transport layer (TCP/IP model)
 - ◇ Establishes two TCP connections
- ◇ Typically used for **trusted** inside users
- ◇ Monitors TCP data packet handshaking and session fulfilment of firewall rules and policies
- ◇ Does **not filter** individual packets
- ◇ Relays TCP segments **without examining** contents
- ◇ Enforce a security function
 - ◇ That determines which connections will be allowed

Stateful Multilayer Inspection Firewalls

- ◆ Provide the functionality of packet filtering, circuit-level and application-level firewalls combined
- ◆ Can filter traffic at the OSI network layer with ACLs
- ◆ Can filter traffic at the OSI Transport layer by monitoring TCP/UDP sessions
- ◆ Can also filter traffic at the OSI application layer based on application or service
- ◆ They are powerful but also the most expensive type of network firewalls

Vulnerability

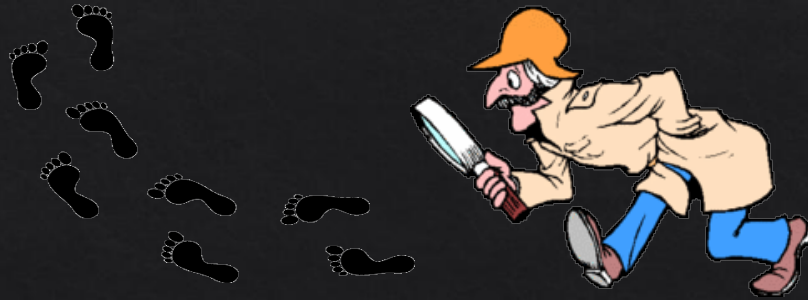
- ◇ What is a vulnerability?
 - ◇ Software flaws or configuration error(s) that allow attackers to violate the security objectives (CIA) of the system
- ◇ Vulnerabilities arise from bugs in applications or design flaws in the system
- ◇ Vulnerabilities can be found in all layers of the system

Vulnerability Scanning

- ◇ Vulnerability scanning can
 - ◇ Identify **flaws** in the system applications and designs
 - ◇ Find what **services** are running
 - ◇ Ensure vulnerabilities are **patched** correctly
 - ◇ Part of a pre-emptive step to **remove** vulnerabilities before attackers exploit them

Vulnerability Scanning

- ❖ “Scanning” is means of **collecting information** about computer systems and the entities they belong to
- ❖ Attackers also conduct “scanning” in the pre-attack phase, or more commonly known as **reconnaissance**



Vulnerability Scanning

- ◇ How are vulnerabilities discovered?
 - ◇ Known vulnerabilities using tools
 - ◇ New vulnerabilities by humans
 - ◇ Hacker groups
 - ◇ Security companies or
 - ◇ Researchers
 - ◇ E.g., a group may discover a vulnerability related to a software (could be by accident or through a directed research)
 - ◇ Vulnerability is disclosed to the public
 - ◇ **When?**

Vulnerability Scanning

- ◇ How are vulnerabilities discovered?
 - ◇ Known vulnerabilities using tools
 - ◇ New vulnerabilities by human
 - ◇ Hacker groups
 - ◇ Security companies or
 - ◇ Researchers
 - ◇ E.g., a group may discover a vulnerability through directed research)
 - ◇ Vulnerability is disclosed to the public
 - ◇ **When?**

← Blog / News

Claude Opus 4.6 found 500 vulnerabilities. What does this change for software security?

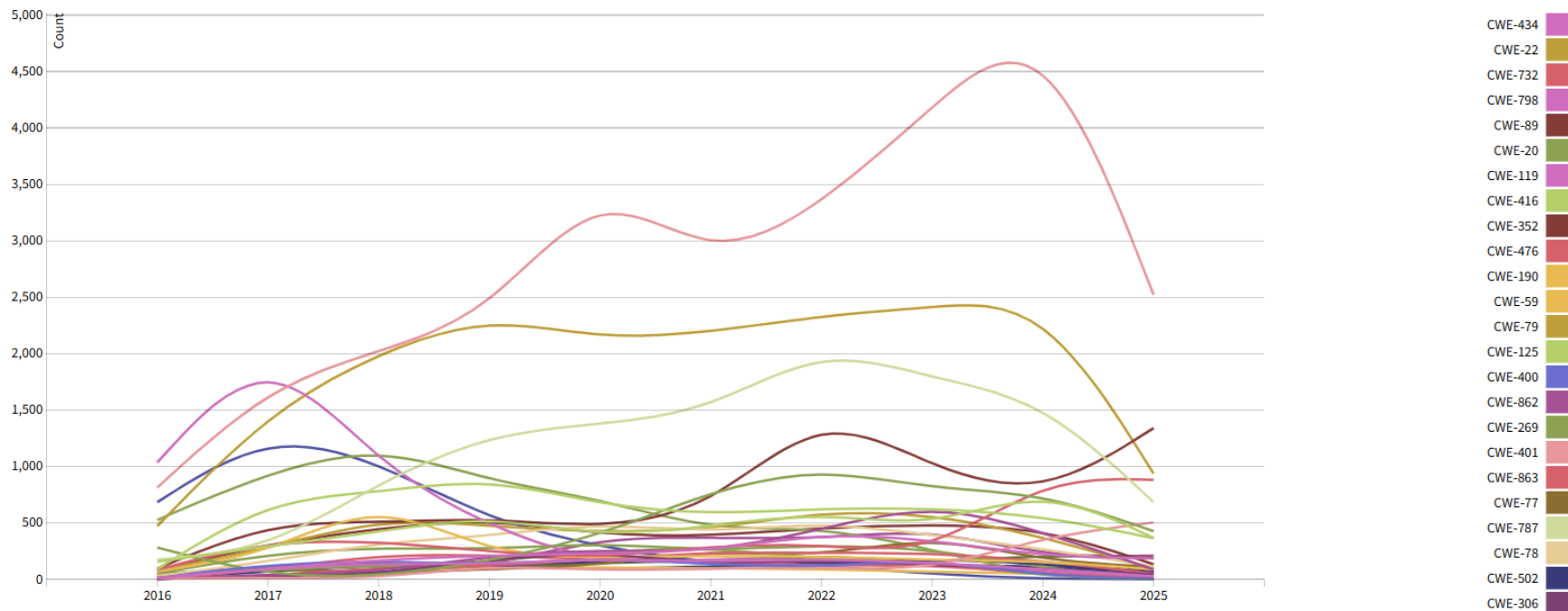
Written by [Sooraj Shah](#)

Published on: Feb 11, 2026

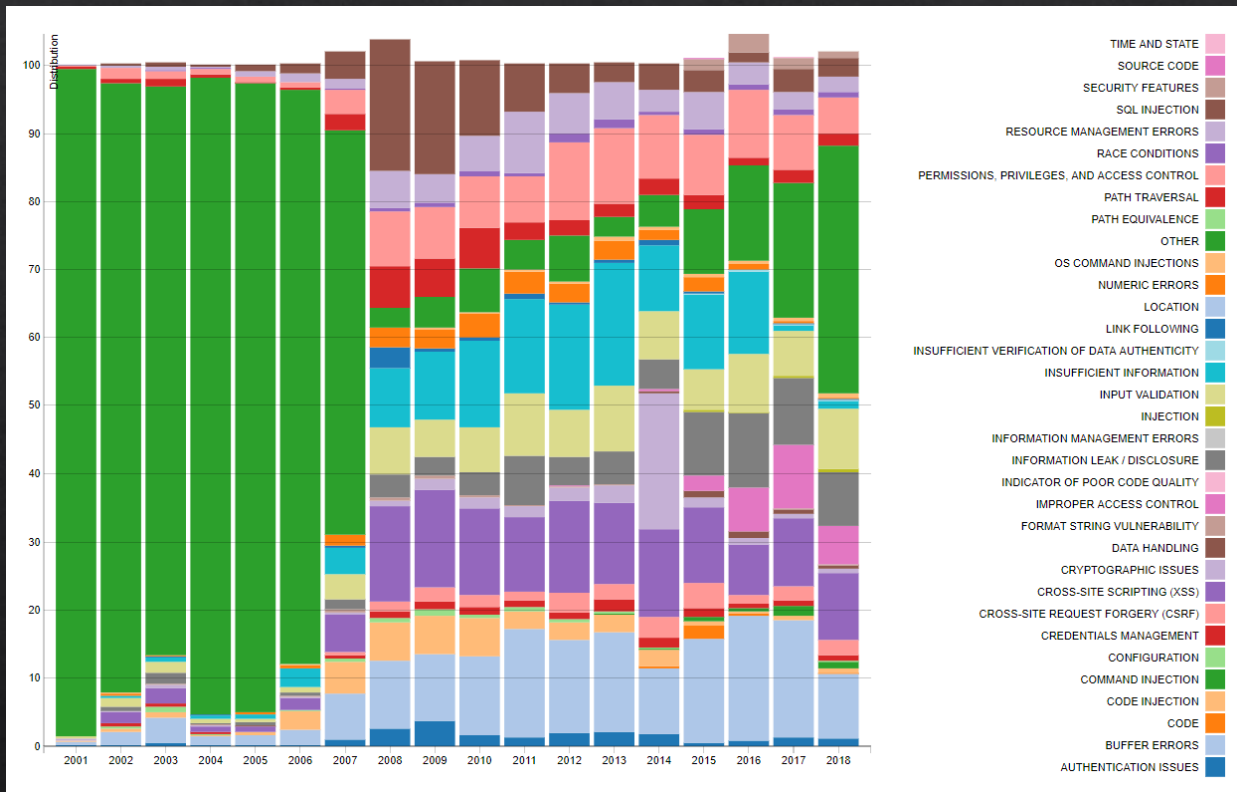
Vulnerability Scanning

Vulnerability Type Change By Year

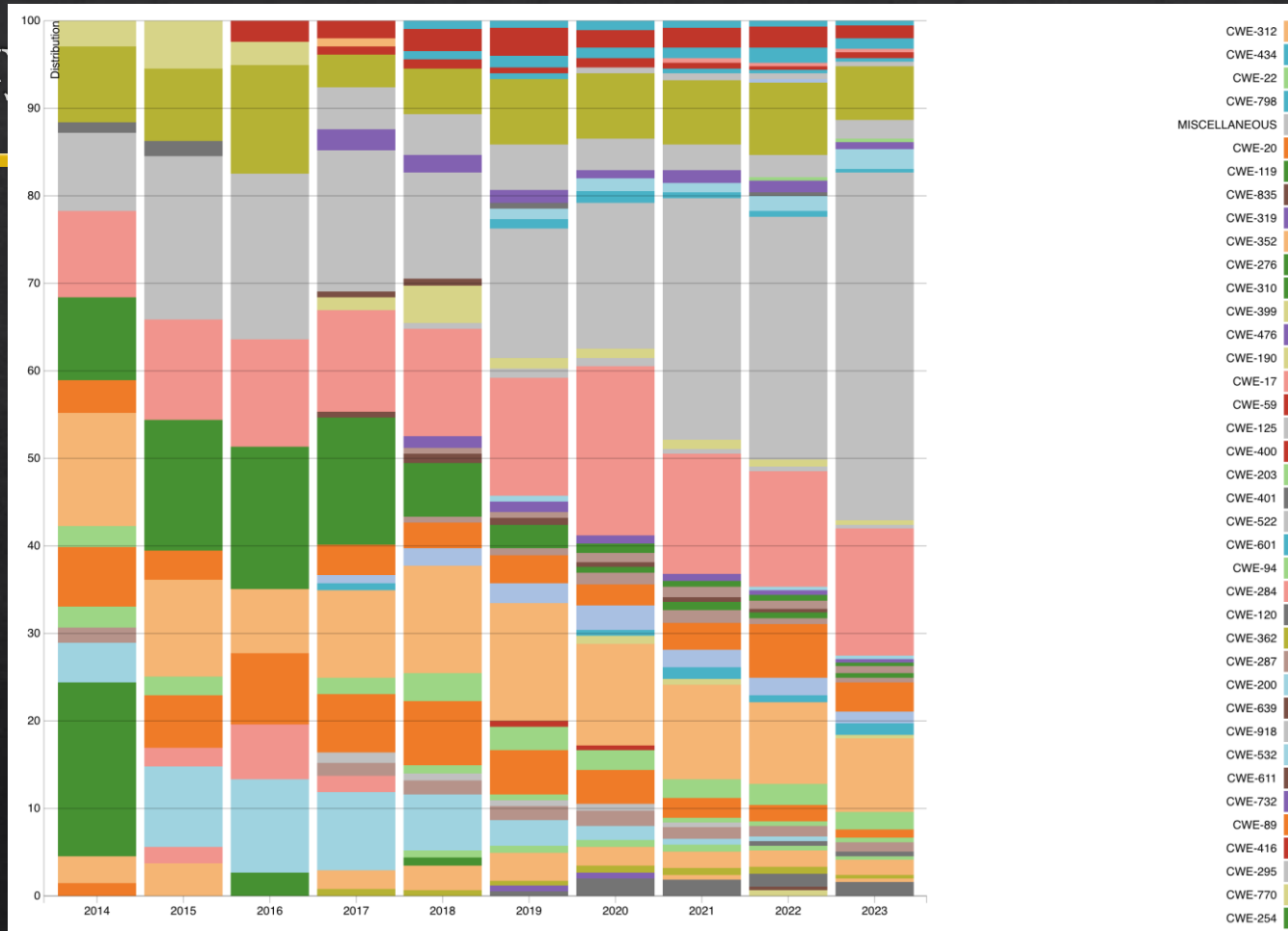
This visualization is a slightly different view that emphasizes how the assignment of CWEs has changed from year to year.



Vulnerability Scanning



Vulnerability



Vulnerability Scanning

- ◇ Some widely used scanning categories are

- ◇ Port scanners

- ◇ Nmap – <https://nmap.org/>

- ◇ Network vulnerability scanners

- ◇ Nessus - <https://www.tenable.com/products/nessus/nessus-professional>

- ◇ OpenVAS - <http://www.openvas.org/>

- ◇ Qualys, SAINT, etc.

- ◇ Web application vulnerability scanning tools

- ◇ List from OWASP - https://www.owasp.org/index.php/Category:Vulnerability_Scanning_Tools

- ◇ There are other types as well

- ◇ E.g., database security scanner, host-based vulnerability scanner etc.

Basics of scanning

- ◇ Ping and ICMP (Internet Control Message Protocol) scan
 - ◇ Typically for server scanning
- ◇ TCP and UDP scan
 - ◇ Typically for port scanning

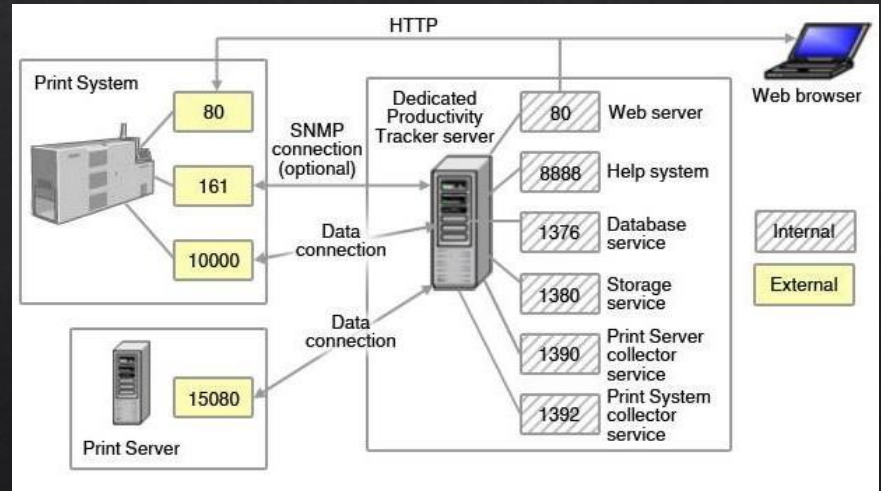
Ping

```
→ ~ ping www.uwa.edu.au
PING cf05-az.uwa.edu.au (104.18.4.186): 56 data bytes
64 bytes from 104.18.4.186: icmp_seq=0 ttl=51 time=26.610 ms
64 bytes from 104.18.4.186: icmp_seq=1 ttl=51 time=18.525 ms
64 bytes from 104.18.4.186: icmp_seq=2 ttl=51 time=7.321 ms
64 bytes from 104.18.4.186: icmp_seq=3 ttl=51 time=6.380 ms
^C
--- cf05-az.uwa.edu.au ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 6.380/14.709/26.610/8.369 ms
→ ~
```

- 1) ICMP packet length
- 2) Time to receive the ICMP echo reply packet
- 3) Time to live value
- 4) Number of request, reply and lost packets
- 5) Approximate round trip time

Port scanning

Perth Air“port”



Well known ports

Port Number	Protocol	Application
20	TCP	FTP data
21	TCP	FTP control
22	TCP	SSH
23	TCP	Telnet
25	TCP	SMTP
53	UDP, TCP	DNS
67, 68	UDP	DHCP
69	UDP	TFTP
80	TCP	HTTP (WWW)
110	TCP	POP3
161	UDP	SNMP
443	TCP	SSL

TCP/UDP scanning

◆ UDP scanning

- ◆ Attacker sends an UDP packet
 - ◆ No response → the port is open
 - ◆ ICMP Unreachable packet received → the port is closed

◆ TCP scanning

- ◆ Attacker sends SYN packet
 - ◆ SYN+ACK packet received → the port is open
 - ◆ RST+ACK packet received → the port is closed

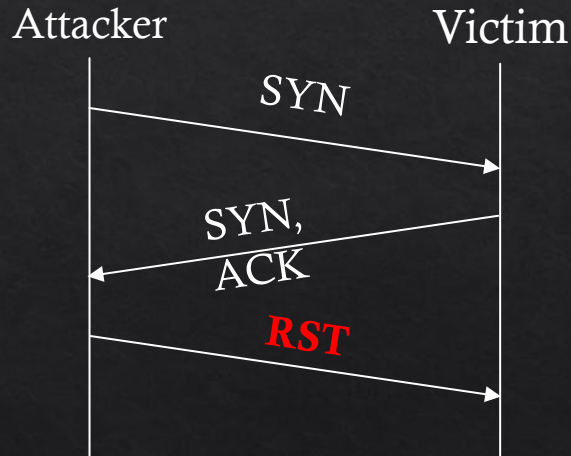
TCP scanning

- ◇ Initiate the 3-way handshake to check the port
- ◇ Provides **fast** port scanning and requires **no privilege**
- ◇ However, this is easily **detectable**
 - ◇ And so likely to be **blocked**
- ◇ Sol: use **stealth** scanning methods

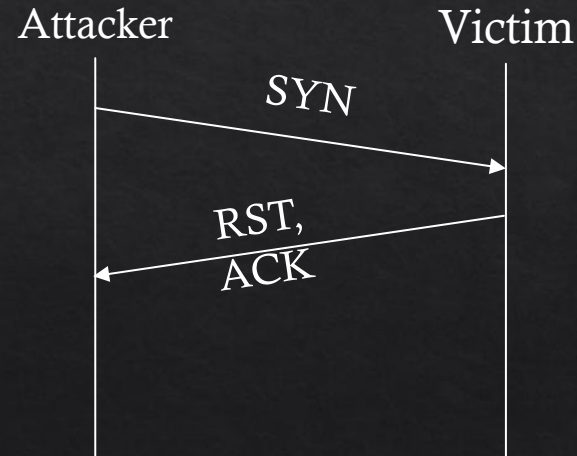
Stealth TCP scanning 1

◇ TCP SYN scanning (aka half-open scanning)

Port is open



Port is closed



Note: many systems will not log this connection attempt

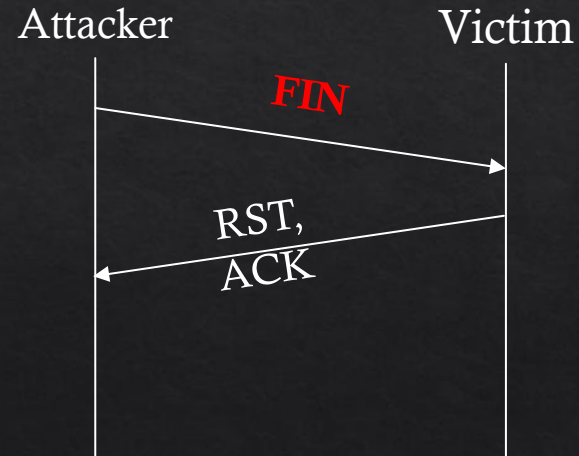
Stealth TCP scanning 2

◇ TCP FIN scanning

Port is open



Port is closed

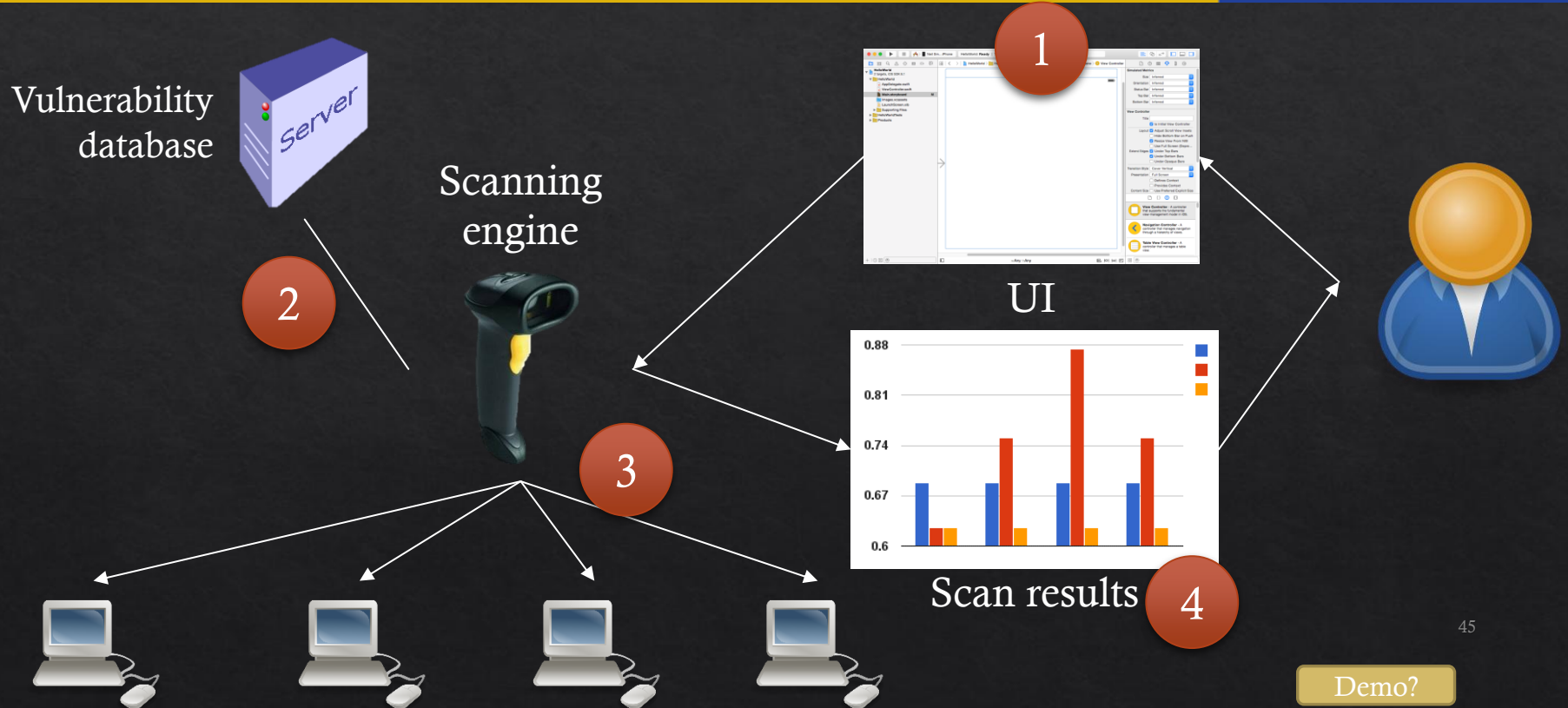


Note: Some systems may sent RST packets regardless

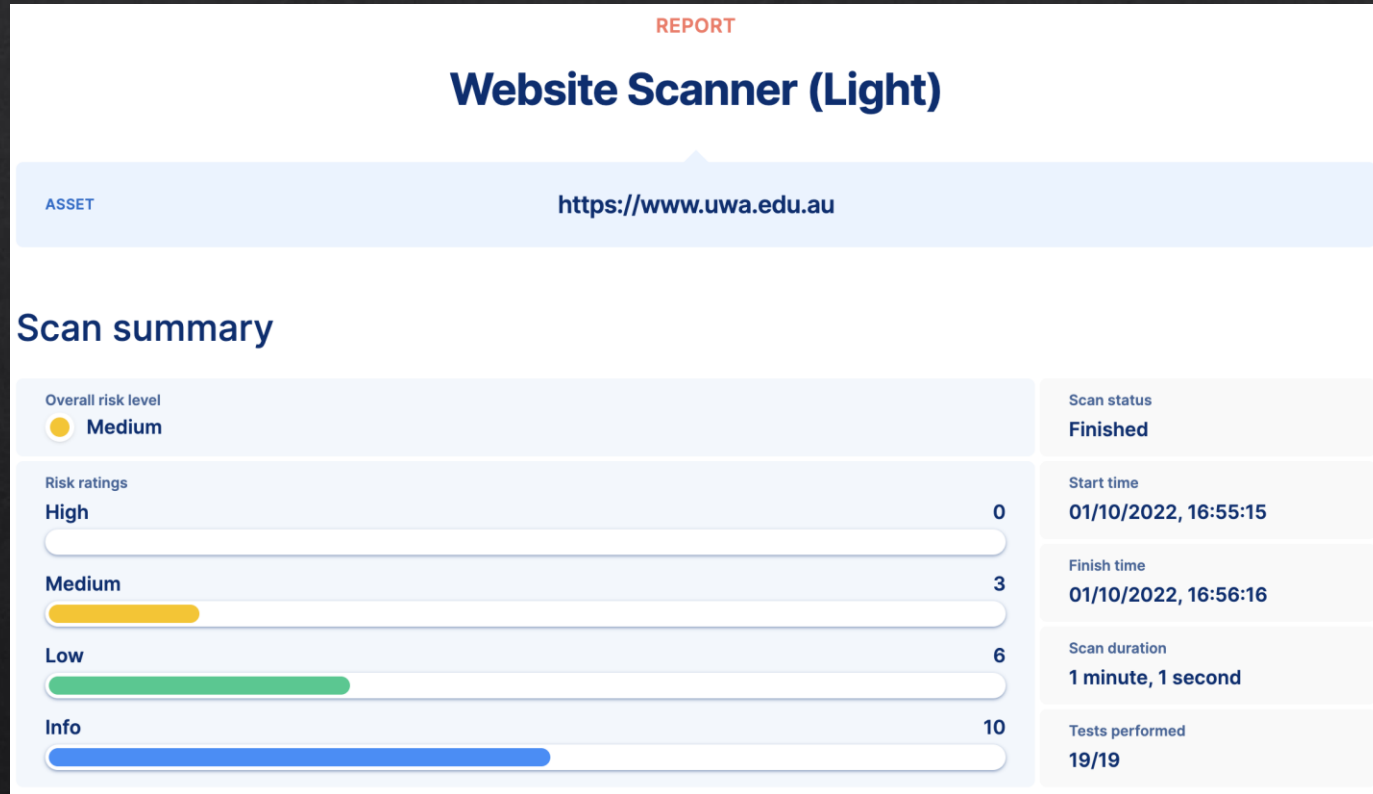
Stealth TCP scanning +

- ◇ Other techniques include
 - ◇ Fragmentation scanning
 - ◇ Chopping the existing scanning packets into smaller fragments
 - ◇ TCP reverse ident scanning
 - ◇ Ident protocol to extract connection information
 - ◇ FTP bounce attack
 - ◇ Manipulating the FTP server protocol interpreter to redirect files
 - ◇ Etc... see *additional items* for more.

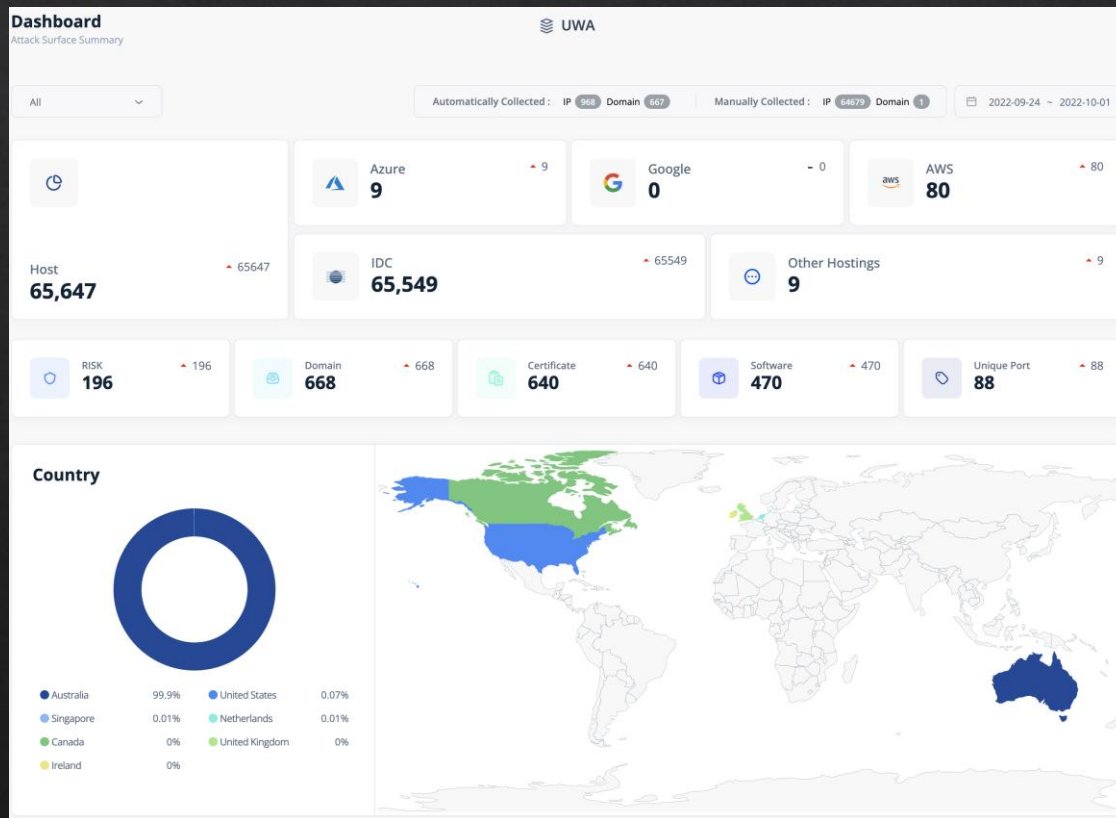
Vulnerability Scanning



Vulnerability Scanning



Vulnerability Scanning



Example: Nessus

- ◇ From Tenable©
- ◇ Runs on various Oses
 - ◇ Linux, Unix-based, Windows etc
- ◇ Keeps their own vulnerability database
 - ◇ Good or bad?
- ◇ Uses Nessus Attack Scripting Language
 - ◇ So you can write your own scripts and plugins
- ◇ There are various (and many free) plugins available to use
 - ◇ <https://www.tenable.com/plugins/nessus/families>
- ◇ Vulnerabilities found are classified based on their risk-factor
- ◇ Is a commercial product
 - ◇ In 2018: \$2300USD per annum for the professional version
 - ◇ In 2019: \$2800USD per annum
 - ◇ In 2024: \$4389USD per annum!
 - ◇ In 2026: \$ **5,269USD** per annum!



Example: Nessus

- ◇ Nessus can scan various types of vulnerabilities
 - ◇ Windows vulnerabilities
 - ◇ Insecure scripts
 - ◇ E.g., Common Gateway Interface
 - ◇ RPC (remote procedure call) program vulnerabilities
 - ◇ Firewall misconfigurations
 - ◇ FTP insecure implementations
 - ◇ Etc.
- ◇ Provides ranking of them

128.143.13.168

Summary

Critical0

High0

Medium4

Low1

Details

Severity	Plugin Id	Name
Medium (5.1)	18405	Microsoft Windows Remote Desktop Protocol
Medium (5.0)	26920	Microsoft Windows SMB NULL Session Authentication
Medium (5.0)	57608	SMB Signing Disabled
Medium (4.3)	57690	Terminal Services Encryption Level is Medium
Low (2.6)	30218	Terminal Services Encryption Level is not Fully Enabled
Info	10150	Windows NetBIOS / SMB Remote Host Information
Info	10287	Traceroute Information
Info	10394	Microsoft Windows SMB Log In Possible
Info	10785	Microsoft Windows SMB NativeLanManager Authentication
Info	10940	Windows Terminal Services Enabled

Example: CriminalIP

The screenshot displays the CriminalIP website's search interface. At the top, the navigation bar includes the CriminalIP logo, links for Search Engine, Products, Resources, About, and Contact Us, a globe icon, a Pricing button, and a user profile icon labeled 'J'. The main content area features a large heading: "Search for information on **certificates** connected to the public Internet." Below this, a circular diagram on the left illustrates various asset types: Exploit, Hacking Group, Image, and Domain. To the right of the diagram, a filter bar shows "Top 10", "3 Keyword webcamxp", and "3 IP 185.189.182.234". A central search bar with a magnifying glass icon and the label "Asset" contains the text "Try to search assets with the following filter examples below". A blue arrow button is positioned to the right of the search bar. Below the search bar, a button labeled "Look up my IP address" is visible. The background of the page features a faint world map and a network diagram.

CriminalIP

Search Engine ▾ Products ▾ Resources ▾ About ▾ Contact Us ▾

Pricing J ▾

Search for information on **certificates** connected to the public Internet.

Exploit

Hacking Group

Image

Domain

Top 10 3 Keyword **webcamxp** 3 IP **185.189.182.234**

Asset Try to search assets with the following filter examples below

Look up my IP address

Example: CriminalIP

- ◇ IP-based threat detection tool
- ◇ Scans the whole internet regularly to update its data
- ◇ Using AI, determine the vulnerabilities and threats detected based on network traffic

Example: Criminal

Asset Search

Showing results for

Keyword : **commonwealth bank of**

Filter : **none** (You can add multiple filters)

8.217.136.79:80 [🔗](#)

Inbound **Dangerous**

Outbound **Moderate**

200

nginx

Alibaba US Technology Co., Ltd.

Hong Kong

Hong Kong

2024-04-05 03:48:01

Asset Search

79.124.78.64



79.124.78.64

IP Scoring

Inbound: **Critical**



Outbound: **Moderate**



- ✓ This is a malicious IP Address.
- 👁 You first viewed this IP address.
- ⚠ This IP Address has critical vulnerabilities.

Current Open Ports

TCP

22 80

⚠ This has vulnerabilities.

UDP

No open port information.

Summary

Connection

Representative Domain	N/A
SSL Certificate	N/A
IP Address Owner	Verdina Ltd.
Hostname	N/A
Connected Domains	1
Country	Bulgaria

Detection

Proxy IP	False
VPN IP	False
Tor IP	False
Hosting IP	False
Mobile IP	False
CDN IP	False
Scanner IP	False
Special Issue	0
Anonymous VPN Detection	False

Security

Abuse Record	0
Open Ports	2
Vulnerabilities	53
Exploit DB	4
Policy Violation	1 (OpenSSH)
Remote Address	True
Network Device	0
Admin Page	False
Invalid SSL	False

Intelligence

Real IP	False
Hacking Group	0

Vulnerability database

- ◇ Widely used vulnerability database



Vulnerability database

- ◇ CVE provides a list of standardised names for vulnerabilities

- ◇ These vulnerabilities are publicly known

- ◇ Security experts form editorial board and provide the description

- ◇ E.g., CVE-2024-3094

Malicious code was discovered in the upstream tarballs of xz, starting with version 5.6.0. Through a series of complex obfuscations, the liblzma build process extracts a prebuilt object file from a disguised test file existing in the source code, which is then used to modify specific functions in the liblzma code. This results in a modified liblzma library that can be used by any software linked against this library, intercepting and modifying the data interaction with this library. – the **XZ backdoor** vulnerability

- ◇ MITRE Corporation maintains CVE and moderates the editorial board discussions



Vulnerability database

- ◇ NVD is a comprehensive cybersecurity vulnerability database
 - ◇ Maintained by NIST, an US organisation
 - ◇ Synchronised with CVE vulnerability naming standard
 - ◇ Provides some security metrics about the vulnerability
 - ◇ E.g., CVSS base score, vectors, etc.
 - ◇ CVE-2024-3094 has the CVSS BS of 10.0!



Vulnerability Scanning: Summary

- ◆ Requires continuous efforts to discover and mitigate vulnerabilities
- ◆ Difficult to address problems with new and unknown vulnerabilities
 - ◆ E.g., zero-day vulnerabilities
- ◆ Global efforts are made to address issues associated with each steps of vulnerability scanning
 - ◆ E.g., improve anomaly detection using AI and machine learning

Sandboxing

- ◇ Often we have untrusted code that we wish to run
 - ◇ E.g., program from Internet including toolbars, viewers, codecs etc
- ◇ We should contain that code in a zone such that if it misbehaves, we can easily kill it
- ◇ Create the zone through confinement – sandboxing!



Sandboxing



- ◆ **Confinement**: To contain the application in an isolation to ensure it does not carry out unapproved actions
- ◆ Can be implemented at many levels:
 - ◆ **Hardware**: run application on isolated hw (air gap)
 - ◆ **Virtual machines**: isolate OS's on single hardware
 - ◆ **System call interposition**: Isolates a process in a single operating system
 - ◆ **Software Fault Isolation (SFI)**: Isolating threads sharing same address space
 - ◆ **Application specific**: e.g. Windows Sandbox (WSB) or browser-based confinement

System Call Interposition

- ◆ **System Call Interposition:** It is a technique that intercepts and monitors or modifies the system calls made by a process before they reach the operating system kernel
 - ◆ It works by acting as a mediator between an application and the kernel, allowing for the enforcement of fine-grained security policies based on the type and arguments of a system call (e.g., permitting `open()` calls only for specific files or directories).
 - ◆ It's used for various purposes including security monitoring, enforcing access control, program tracing, and even creating portable software packages by capturing all dependencies.
- ◆ **Limitation:** Interposition tools can incur performance overhead and may be bypassed if not implemented carefully to cover all possible paths to the kernel

chroot



- ◇ Example: using *chroot* (short for change root)
- ◇ Often used for guest accounts
- ◇ Below sets a low-privilege user inside a restricted filesystem

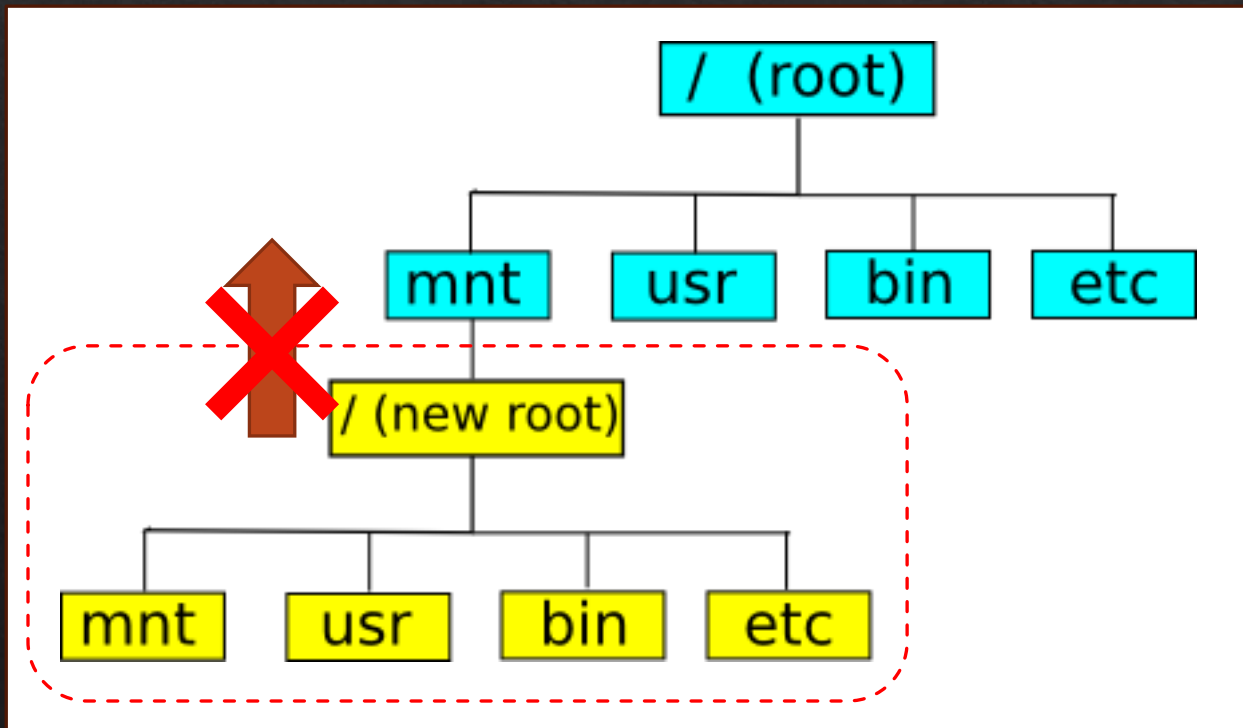
Requires root privilege

<code>chroot /tmp/guest</code>	root dir “/” is now “/tmp/guest”
<code>su guest</code>	EUID is set to “guest”

- ◇ The path “/tmp/guest” is added to the file system accesses for applications in jail

Problem: utility programs also need to be available in the jail (e.g., `ls`, `ps`, `vi`)

chroot



Access is restricted
within the boundaries



chroot



- ◊ Application cannot access files outside of jail
- ◊ Jailkit is used to address this problem
- Automatically builds files, libs, and directories needed in the jail environment
 - **jk_init**: creates jail environment
 - **jk_check**: checks jail env for security problems
 - checks for any modified programs,
 - checks for world writable directories, etc.
 - **jk_lsh**: restricted shell to be used inside jail

chroot



◇ Of course, people will try to break out of jails...

Scenario 1:

```
open("../etc/password", "r")
```

Scenario 2:

```
mkdir "/asdf/etc/passwd"
```

```
chroot "/asdf"
```

```
chroot("../..../..../..../..../..../..../..../..../..")
```

If the process has a root privilege, you can escape the jail!

chroot



- ◇ chroot **lacks** many capabilities
 - ◇ All or nothing access to the file system
 - ◇ Not so useful for web applications etc
 - ◇ Need to provide access to files outside the jail
 - ◇ Malicious applications can access the network
 - ◇ And do something with other internally connected machines
- ◇ Suggestion: **do not** use chroot as a security solution (It's just a filesystem view change, not a sandbox)
 - ◇ Use better approaches and combinations with other approaches

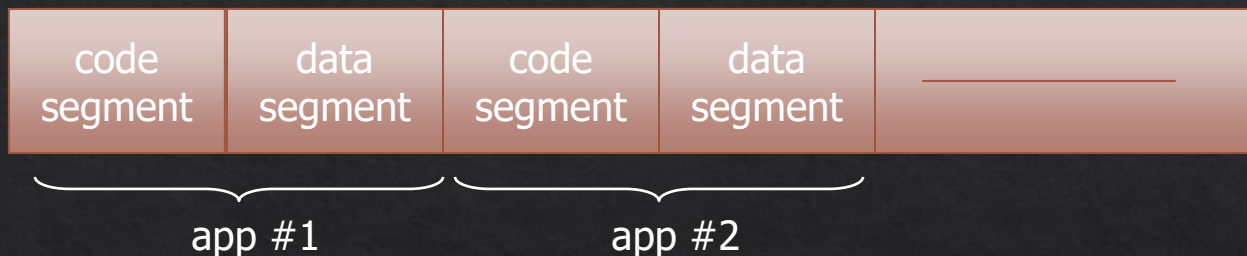
SFI

- ◇ Software Fault Isolation
- ◇ **Problem**: legitimate and malicious applications running in the same address space
 - ◇ E.g., device drivers trying to corrupt kernel
- ◇ **Solution**
 - ◇ Separate applications into different address spaces
 - ◇ Instructions are added before memory operations and verify behaviour (e.g., illegal memory access)



SFI

- ◆ Partition process memory into segments



- ◆ Locate unsafe instructions (e.g., jmp, load, store)
 - ◆ Add safety instructions for memory access
 - ◆ Add guards before unsafe instructions at compile time
 - ◆ Validate guards when loading them

SFI

- ◇ Reconstruct code instructions for safety
- ◇ Untrusted code (applications) should only be able to
 - ◇ Jump within its domain's code segment
 - ◇ Write within its domain's data segment

Unsafe instruction

```
STR R0, R1      ; write R1 to Mem[R0]
```

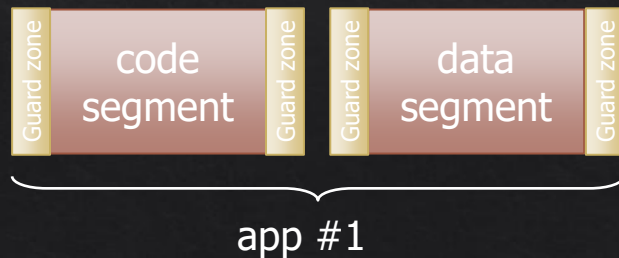
Rewritten instruction by sandbox

STR – store
MOV – move
SHR – shift logical right
CMP – compare
BNE – branch on not equal

```
MOV  Ra, R0      ; copy R0 into Ra  
SHR  Rb, Ra, Rc   ; Rb = Ra >> Rc, get the segment ID  
CMP  Rb, Rd       ; Rd holds the data segment ID  
BNE  fault        ; wrong data segment ID  
STR  Ra, R1       ; Ra in data segment, so write
```

SFI

- ◇ Some instructions use register and offset addressing
 - ◇ i.e., needs extra space for the instruction
- ◇ Add guard zone to each segment to avoid calculating the offset



SFI

- ◇ Verifier ensures **all** instructions are safe
- ◇ Verifier checks **no privileged** instructions are in the code
- ◇ Verifier also checks relevant instructions are **within** the code/data segments
- ◇ If the sandboxed code **fails** any checks, verifier *rejects* the code



SFI

- ◇ Provides a **good** performance with a little overhead
 - ◇ Typically around 4% processing overhead
- ◇ **Confines** writes and control transfers in extension's data and code segments, respectively
- ◇ Prevent execution of **privileged** instructions
- ◇ However, more **difficult** to implement on x86 architectures
 - ◇ It was not designed for x86 in the first place
 - ◇ Variable length instructions: hard to place guards
 - ◇ Many instructions that affects the memory: need more guards

Sandboxing summary

- ◇ Various techniques for sandboxing in different layers of the system
- ◇ It is difficult to fully isolate malicious code, due to dependencies
- ◇ Defining sandboxing policy is one of the top key challenges



Other cool tools

- ◇ There are many other tools that are useful for defence (as well as attacks), such as
 - ◇ Wireshark
 - ◇ Metasploit
 - ◇ Snort
 - ◇ Burp
 - ◇ OpenVAS
 - ◇ Etc.

Summary

- ◇ We looked at security related tools and methods
 - ◇ Firewall
 - ◇ vulnerability scanning
 - ◇ sandboxing
- ◇ There are **many other** security methods and techniques to be explored!
- ◇ All security methods and techniques provide means to **ensure the security objectives** of the system
- ◇ Should be carefully **designed and implemented** to maximise the security

Additional Items

- ◊ Firewall
 - ◊ <https://www.digitalocean.com/community/tutorials/what-is-a-firewall-and-how-does-it-work>
- ◊ List of security tools
 - ◊ <https://sectools.org/> (Top 125 Network Security Tools)
 - ◊ [https://owasp.org/www-community/Free for Open Source Application Security Tools](https://owasp.org/www-community/Free_for_Open_Source_Application_Security_Tools) (Free for Open Source Application Security Tools)
- ◊ Port scanning
 - ◊ https://nmap.org/nmap_doc.html
- ◊ Vulnerability database
 - ◊ CVE – <https://cve.mitre.org>
 - ◊ NVD – <https://nvd.nist.gov>
- ◊ Sandboxing
 - ◊ chroot jail escape – <http://www.ouah.org/chroot-break.html>